



INSTITUTO POLITÉCNICO NACIONAL

UPIICSA

ABSTRACCIÓN Y USO DE DATOS

EQUIPO: SYNTAX ERROR

3CM32

REPORTE PRÁCTICA 1

Desarrollo: Como el ejercicio 12, pero dentro de la clase o estructura considerando un puntero o referencia al mismo tipo de dato para lograr relaciones de auto-referencia.

Errores:

- Error 1: Dejar el puntero autorreferenciado sin inicializar, provocando que apunte a direcciones de memoria basura.
- Error 2: Intentar declarar un miembro de objeto directo del mismo tipo dentro de la clase en lugar de un puntero, generando un tamaño infinito.

Soluciones:

- Solución 1: Inicializar siempre los punteros internos a nullptr dentro del constructor de la clase o estructura.
- Solución 2: Utilizar obligatoriamente un puntero (Tipo*) para permitir la auto-referencia con un tamaño computable en tiempo de compilación.

Código:

```
#include <iostream>
#include <string>
using namespace std;

class Auto {
private:
    float precio;
    int anio;
    Auto* siguiente;

public:
    Auto() {
        precio = 0;
        anio = 0;
        siguiente = nullptr;
    }

    void capturar() {
        cout << "Ingrese el precio del auto: ";
        cin >> precio;
        cout << "Ingrese el anio del auto: ";
```

```
        cin >> anio;
    }

    void mostrar(int numero) const {
        cout << "Auto " << numero << " Precio: $" << precio
             << " | Anio: " << anio << endl;
    }

    void setSiguiente(Auto* s) {
        siguiente = s;
    }

    Auto* getSiguiente() const {
        return siguiente;
    }
};

class Persona {
private:
    string nombre;
    string ap;
    string am;
    string genero;
    int edad;
    Persona* siguiente;

public:
    Persona() {
        edad = 0;
        siguiente = nullptr;
    }

    void capturar() {
        cout << "Ingresa el nombre: ";
        cin >> nombre;
        cout << "Ingresa el apellido paterno: ";
        cin >> ap;
        cout << "Ingresa el apellido materno: ";
        cin >> am;
        cout << "Ingresa el genero: ";
        cin >> genero;
        cout << "Ingresa la edad: ";
        cin >> edad;
    }

    void mostrar(int numero) const {
        cout << "Persona " << numero << " -> " << nombre << " "
             << ap << " " << am
```

```
            << " | Genero: " << genero
            << " | Edad: " << edad << " anios" << endl;
    }

    void setSiguiente(Persona* s) {
        siguiente = s;
    }

    Persona* getSiguiente() const {
        return siguiente;
    }
};

int main() {

    const int TOTAL = 2;

    Auto autos[TOTAL];
    Persona personas[TOTAL];

    cout << "registro autos" << endl;
    for (int i = 0; i < TOTAL; i++) {
        cout << "\nAuto " << (i + 1) << ":" << endl;
        autos[i].capturar();
        if (i > 0) {
            autos[i - 1].setSiguiente(&autos[i]);
        }
    }

    cout << "\n\nregistro persona" << endl;
    for (int i = 0; i < TOTAL; i++) {
        cout << "\nPersona " << (i + 1) << ":" << endl;
        personas[i].capturar();
        if (i > 0) {
            personas[i - 1].setSiguiente(&personas[i]);
        }
    }

    cout << "\n lista " << endl;
    Auto* actualAuto = &autos[0];
    int numAuto = 1;
    while (actualAuto != nullptr) {
        actualAuto->mostrar(numAuto);
        actualAuto = actualAuto->getSiguiente();
        numAuto++;
    }

    cout << "\n lista de personas " << endl;
```

```
    Persona* actualPersona = &personas[0];
    int numPersona = 1;
    while (actualPersona != nullptr) {
        actualPersona->mostrar(numPersona);
        actualPersona = actualPersona->getSiguiente();
        numPersona++;
    }

    return 0;
}
```